

Java 実験レポート 課題 1

課題番号：1

氏名：張 慶志郎

学籍番号：202411005

メールアドレス：s2411005@u.tsukuba.ac.jp

提出年月日：2026 年 5 月 11 日

提出期限日：2026 年 5 月 13 日

課題 1-1-1: クラスとインスタンスの意味をそれぞれ調べ、具体的な例を挙げて説明しなさい。

方針

公式ドキュメントを確認したほか、書籍[1]を用いて意味を調べ、まとめた。

課題の回答

クラス: オブジェクト（インスタンス）を生成するための「設計図」や「ひな型」に相当するもの。インスタンスがどのようなデータ（属性やフィールド）を持ち、どのような振る舞い（メソッド）ができるかの枠組みを定義する。

インスタンス: クラスという設計図に基づいて、コンピュータのメモリ上に実際に生成された「実体」のこと。同じクラスから生成されても、それぞれが独立した情報や状態を持つことができる。

例 1: モンスターが登場するゲームがあるとする。モンスターには複数種類があり、火を吐いたり、魔法を使ったりし、出現する場所や HP（体力）、攻撃方法が異なる。

クラス: 「火を吐くモンスターは HP が 40 で、火山に出現して、このように攻撃する...」のように定義するもの

インスタンス: 個々のモンスター

例 2: 学生の情報を管理するソフトウェアがあるとする。

クラス: 学籍番号や氏名など、「どの情報を管理するのか」を定義するもの

インスタンス: 「学籍番号：202411005、氏名：張 慶志郎」「学籍番号：12345、氏名：筑波 太郎」のような個々の学生情報の実体

課題 1-1-2: クラスの継承について調べ、クラスの継承がどのようなプログラムを書くときに便利なのか、課題のプログラムを例に挙げながら説明しなさい。

方針

書籍^[1]を用いてオブジェクト指向における継承の概念を整理し、本課題で作成した `MyDrawing` クラスのプログラムコードと照らし合わせて具体的な利点を考察した。

課題の回答

クラスの継承とは、既存のクラス（親クラス・スーパークラス）が持つフィールド（データ）やメソッド（処理）を引き継いで、新しいクラス（子クラス・サブクラス）を作成する仕組みのことである。Java においてクラス A を継承したクラス B を作成する際には、以下のように記述する。

```
class B extends A { ...  
  
}
```

継承は、複数のクラス間で共通する属性や処理が存在する場合に、コードの重複を防ぐために非常に便利である。

課題のプログラムを例にとる。最初に作成したクラス `MyDrawing` には、すでに

- ・ 図形を描画する際の始点の座標の定義
- ・ 図形の幅・高さの定義
- ・ 図形の線の太さの定義
- ・ 図形の線の色

が含まれている。（図 1）

今後、様々な図形のクラスを作成するうえで、上のような定義は必要であるが、図形の種類ごとに作成するクラスを新たに追記するたびにこれらを定義するのは大変であり、コードが長くなってしまうことから読みやすさも損なわれる。よって、

```
Class My○○（作成する図形） extends MyDrawing {  
  
}
```

このように記述することで、最低限必要な情報（座標や色の定義など）を記述するコードを再利用できる。子クラスには各図形固有の機能のみを追加すればよくなるため、記述の手間が減り、プログラムの保守性や可読性が大幅に向上する。これがクラスの継承の利点である。

¹ 三谷 純. (2017 年). Java 第 2 版 入門編. 翔泳社. 131-139 項、192-194 項

ソースコード(MyDrawing.java)

```
1 import java.awt.*;
2
3 public class MyDrawing
4 {
5     private int x, y, w, h; // X 座標, Y 座標, 幅, 高さ
6     private Color lineColor, fillColor; // 線の色, 塗り色
7     private int lineWidth; // 線の太さ
8 }
```

図1 MyDrawing.java にすでにある定義

課題 1-2-1: 「楕円」を描くクラスと「正 11 角形」を描くクラス MyHendecagonal を作り、それぞれのインスタンスを作って MyCanvas 上に表示しなさい。また、API の [Graphics](#) クラスの draw*** メソッドを参考に MyDrawing クラスを継承した 1 つ以上の独自の描画クラス My*** クラスを追加しなさい。

実装方針

楕円を描画する MyOval クラスは、Java の Graphics クラスに標準で用意されている drawOval() メソッドを用いて実装する。この関数は、指定した座標を左上とする長方形（バウンディングボックス）に内接する楕円を描画するものである。

一方、正十一角形を描画する MyHendecagonal クラスは、専用の描画メソッドが存在しないため、多角形を描画する drawPolygon() メソッドを用いて実装する。中心座標と幅から半径を算出し、三角関数を用いて各頂点の座標を数学的に計算して結ぶ方針とする。

また、独自の描画クラスとして五芒星を描画する MyfiveStar クラスを追加する。これも正十一角形と同様に、各頂点の座標を計算し drawPolygon() メソッドを用いて描画する方針とした。

実装

ソースコード 1 に示す MyOval クラスでは、描画メソッド内で drawOval(x, y, w, h) を呼び出し、親クラスから取得した座標と幅・高さを渡すことで楕円を描画した。

ソースコード 2 に示す MyHendecagonal クラスでは、まず 2π (360°) を 11 等分した角度（ラジアン）を算出した。親クラスの幅 w の半分の値を半径 r とし、for ループを用いて各頂点の X 座標・Y 座標を Math.cos と Math.sin を用いて計算した。計算した座標をそれぞれ専用の配列に格納し、最後に drawPolygon() に渡すことで正十一角形を描画した。

ソースコード 3: *MyfiveStar.java* ソースコード 3 に示す **MyfiveStar** クラスでも同様の手法を用いた。 $2\pi/5$ を基本の角度 **fiverad** とし、星型の一筆書きになるよう頂点を一つ飛ばしで結ぶため、**for** ループ内で角度を $2*\text{fiverad}*i$ として各頂点の座標を計算した。これらを配列に格納し、**drawPolygon()** を実行して描画した。

最後に、ソースコード 4 に示すように、**MyApplication** クラスにおいて、追加した各図形クラスのインスタンスを生成し、**addDrawing** メソッドを用いてキャンバス (**MyCanvas**) に追加する処理を実装した。

実行結果

MyApplication クラスにて、作成した **MyOval**、**MyHendecagonal**、**MyfiveStar** のインスタンスを生成し、**MyCanvas** に追加してプログラムを実行した。実行結果を図 2 に示す。指定した座標とサイズに基づいて、楕円 ($w=h=40$ のため正円になっている)、正十一角形、五芒星がキャンバス上に想定通りに描画されていることを確認した。

ソースコード 1: *MyOval.java*

```
import java.awt.*;

public class MyOval extends MyDrawing
{
    public MyOval(int xpt, int ypt){
        super();
        setLocation(xpt, ypt);
    }

    public void draw( Graphics g){
        int x = getX();
        int y = getY();
        int w = getW();
        int h = getH();

        Graphics2D g2 = (Graphics2D) g;
        g2.setStroke(new BasicStroke(getLineWidth()));
        g2.setColor(getFillColor());
        g2.fillOval(x, y, w, h);
        g2.setColor(getLineColor());
        g2.drawOval(x, y, w, h);
    }
}
```

ソースコード 2: *MyHendecagonal.java*

```
import java.awt.*;
```

```

public class MyHendecagonal extends MyDrawing
{
    public MyHendecagonal(int xpt, int ypt){
        super();
        setLocation(xpt, ypt);
    }
    public void draw(Graphics g){
        int x = getX();
        int y = getY();
        int r = getW()/2;
        int n = 11;
        int basex = x + r;
        int basey = y + r;
        double elevenrad = 2 * Math.PI / n;

        int xlist[] = new int[n];
        int ylist[] = new int[n];
        for (int i = 0; i < 11; i++){
            xlist[i] = basex + (int) (r * Math.cos(i * elevenrad));
            ylist[i] = basey + (int) (r * Math.sin(i * elevenrad));
        }

        Graphics2D g2 = (Graphics2D) g;
        g2.setStroke(new BasicStroke(getLineWidth()));
        g2.setColor(getFillColor());
        //g2.fillPolygon(xlist, ylist, n);
        g2.setColor(getLineColor());
        g2.drawPolygon(xlist, ylist, n);
    }
}

```

ソースコード3: MyfiveStar.java

```

import java.awt.*;

public class MyfiveStar extends MyDrawing
{
    public MyfiveStar(int xpt, int ypt){
        super();
        setLocation(xpt, ypt);
    }

    public void draw(Graphics g){
        int x = getX();
        int y = getY();
        int r = getW()/2;
        int basex = x + r;
        int basey = y + r;

```

```

int n = 5;
double fiverad = 2 * Math.PI / n;

int xlist[] = new int[n];
int ylist[] = new int[n];

for (int i = 0; i < 5; i++){
    xlist[i] = basex + (int) (r * Math.cos(fiverad* 2 * i - Math.PI/2));
    ylist[i] = basey + (int) (r * Math.sin(fiverad* 2 * i - Math.PI/2));
}

Graphics2D g2 = (Graphics2D) g;
g2.setStroke(new BasicStroke(getLineWidth()));
g2.setColor(getFillColor());
g2.setColor(getLineColor());
g2.drawPolygon(xlist, ylist, n);
}
}

```

ソースコード 4:MyApplication.java の追記部分

```

MyCanvas canvas = new MyCanvas();
for (int i = 0; i<8; i++){
    canvas.addDrawing(new MyHendecagonal(40 + i*40, 40+ i*40));
    canvas.addDrawing(new MyOval(80 + i*80, 40 + i*40));
    canvas.addDrawing(new MyRectangle(40 + i*40, 40+ i*40));
    canvas.addDrawing(new MyfiveStar(40 + i*40, 40+ i*40));
}

```

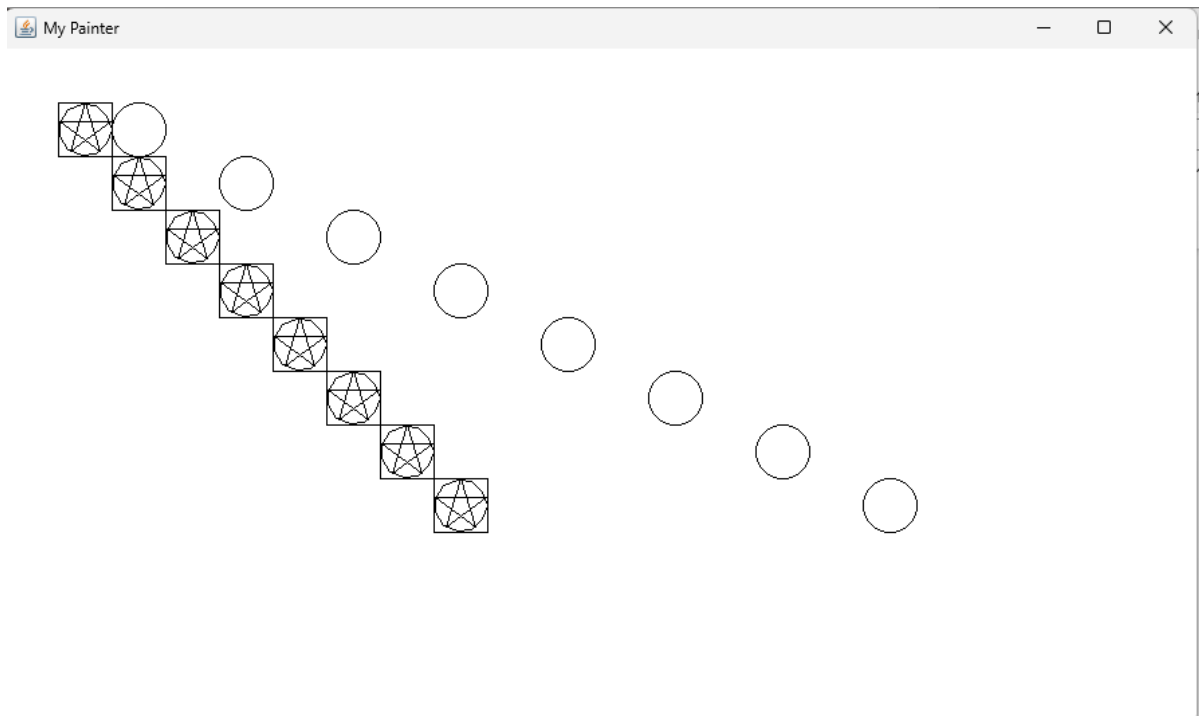


図2 実行のようす

(実行時の解像度では正 11 角形と円との違いが分かりにくいためずらして描画している)

課題 1-2-2: 図形の色や大きさを自由に指定して図形のインスタンスを生成することができるように、図形を描画するクラスのコンストラクタを追加、変更しなさい。コンストラクタは引数の型や数を変えることにより、複数用意することができる。ただし、同じコードの繰り返しにならないように（今後、新しい図形を描画するクラスを追加する際に、最低限の記述で済むように）注意しなさい。

実装方針

図形生成時に座標や色などを指定できるようにするため、親クラスである `MyDrawing` クラスに、引数の型や数が異なる複数のコンストラクタ（オーバーロード）を用意する。各変数の初期化や代入処理は親クラスに集約する。子クラスである `MyHendecagonal` 等では、独自のコンストラクタを定義し、受け取った引数を `super()` を用いて親クラスのコンストラクタへ渡す設計とする。これにより、今後新たな図形クラスを追加する際も、子クラス側は最低限の記述（`super()` の呼び出しのみ）で済むようにし、コードの重複を防ぐ。

実装

ソースコード 5 に示すように、**MyDrawing** クラスに、引数 2 つ（座標）、引数 4 つ（座標とサイズ）、引数 5 つ（座標、サイズ、色）などを受け取る複数のコンストラクタを追加した。これらのコンストラクタ内では、**this.x = x;** のように引数で受け取った値を自身のフィールドに代入する処理を記述した。

ソースコード 6 に示すように、子クラスである **MyHendecagonal** クラスでは、引数を受け取るコンストラクタ内で **super(xpt, ypt, linecolor);** のように記述し、親クラスへ処理を委譲する形で実装した。

実行結果

ソースコード 7 に示すように、テスト用の **MyApplication** クラスにて、引数付きのコンストラクタを利用してインスタンスを生成し、キャンバスに追加した。座標、サイズ、色についてそれぞれ別のコンストラクタを経由して指定できているか確認するために、

(new MyHendecagonal(40 + i*40, 40 + i*40)) : 座標のみ

(new MyHendecagonal(40 + i*40, 40 + i*40, 50, 50)) : 座標とサイズ（幅と高さ）

(new MyHendecagonal(80 + i*80, 80 + i*80, 60, 60, Color.blue)) : 座標とサイズと線の色

のように異なる引数を渡して for ループ内で実行した。

プログラムを実行した結果、コンストラクタで指定した座標や線の色（青）の設定が正しく引き継がれ、キャンバス上に複数の図形が想定通りに描画されることを確認した。（図 3）

ソースコード 5: MyDrawing.java に追記したコンストラクタ

```
public MyDrawing(){
    x = y = 0;
    w = 40;
    h = 40;
    lineColor = Color.black;
    fillColor = Color.white;
    lineWidth = 1;
}

public MyDrawing(int x, int y){
    this.x = x;
    this.y = y;
    w = 40;
    h = 40;
}

public MyDrawing(int x, int y, int w, int h){
    this.x = x;
    this.y = y;
```

```

        this.w = w;
        this.h = h;
    }

    public MyDrawing(int x, int y, int w, int h, Color lineColor){
        this.x = x;
        this.y = y;
        this.w = w;
        this.h = h;
        this.lineColor = lineColor;
    }

```

ソースコード 6:MyHendecagonal.java に追記したコンストラクタの記述

```

    public MyHendecagonal(int xpt, int ypt, int w, int h){
        super(xpt, ypt, w, h);
    }

    public MyHendecagonal(int xpt, int ypt, int w, int h, Color linecolor){
        super(xpt, ypt, w, h, linecolor);
    }

```

ソースコード 7:MyApplication.java に追記・修正した描画記述

```

    MyCanvas canvas = new MyCanvas();
    for (int i = 0; i<8; i++){
        canvas.addDrawing(new MyRectangle(40 + i*40, 40+ i*40));
        canvas.addDrawing(new MyHendecagonal(40 + i*40, 40 + i*40, 50, 50));
        canvas.addDrawing(new MyHendecagonal(80 + i*80, 80+ i*80, 60, 60,
Color.blue));
    }
    jp.add(BorderLayout.CENTER, canvas);

```

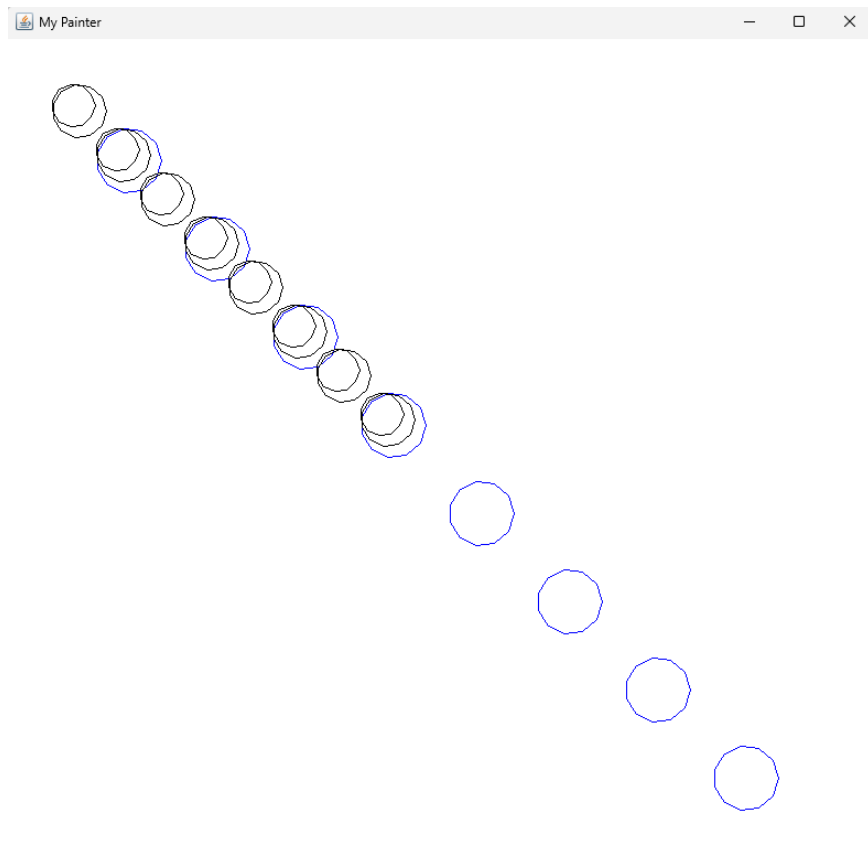


図3:座標、色、サイズがそれぞれ自由に指定したとおりに生成、描画されている様子

課題 1-2-2: 今後の課題でボタンやメニュー、マウスイベント等を使って MYDRAWING のインスタンスの属性を変更するプログラムを書く予定である。MYDRAWING のインスタンスの、大きさや色、線の太さ、線種（実線と破線）などの属性を変更・取得するためのメソッド（アクセッサ）を追加しなさい。

実装方針

クラス MyDrawing にすでに用意されている int 型変数 linewidth、Color 変数の linecolor、fillcolor を用いてアクセッサを追記する。また、描画する線が破線か実線かを指定する変数 dashed、幅・高さではなく一括でサイズを指定できる変数 size を追加し、この属性を取得するためのアクセッサも追記する。

実装

ソースコード 9 に示すように、クラス `MyDrawing` の先頭で変数を追記し、その後ろでソースコード 9 に示すように複数のアクセッサを追記した。どのアクセッサも、`get`~で値を取得し、対応する `set`~で取得した値を返している。

実行結果

この課題では定義・追加のみを行ったため、実行はしていない。

ソースコード 8: `MyDrawing.java` に追記した変数

```
private int x, y, w, h, size;
private Color lineColor, fillColor;
private int lineWidth;
private boolean dashed;
```

ソースコード 9: `MyDrawing` に追記した複数のアクセッサ

```
public void draw (Graphics g){
}

public void move(int dx, int dy){
    x += dx;
    y += dy;
}

public void setLocation(int x, int y){
    this.x = x;
    this.y = y;
}

public void setSize(int size){
    this.size = size;
}

public void setLineColor(Color lineColor){
    this.lineColor = lineColor;
}

public void setFillColor(Color fillColor){
    this.fillColor = fillColor;
}

public void setLineWidth(int lineWidth){
    this.lineWidth = lineWidth;
}
```

```
public void setDashed(boolean dashed){
    this.dashed = dashed;
}

public int getX(){
    return x;
}

public int getY(){
    return y;
}

public int getW(){
    return w;
}

public int getH(){
    return h;
}

public int setSize(){
    return size;
}

public Color getFillColor(){
    return fillColor;
}

public Color getLineColor(){
    return lineColor;
}

public int getLineWidth(){
    return lineWidth;
}

public boolean isDashed(){
    return dashed;
}
```

参考資料

1. 三谷 純.(2017 年). Java 第 2 版 入門編. 翔泳社.131-139 項、192-194 項